

Relatório Técnico

A Aurora Estabelece os Primeiros Sistemas da Colônia

Capítulo 1 — Sistema Inteligente de Análise e Decisão da Colônia Aurora Siger

1. Introdução

Este relatório descreve o sistema desenvolvido para a colônia Aurora Siger, que entrou em sua fase de operação contínua em Marte. Devido à latência de comunicação com a Terra, os sistemas da colônia precisam operar de forma autônoma, interpretando dados dos sensores e tomando decisões sem intervenção humana direta.

O sistema integra os conceitos trabalhados ao longo de toda a fase — lógica booleana, subalgoritmos, estruturas avançadas (hash e árvore), vetores como listas, ciclo buscar–decodificar–executar, regressão linear e geração de energia renovável — em uma solução única construída inteiramente em **Python puro**, sem bibliotecas externas, conforme as observações do desafio.

2. Conceitos da fase aplicados ao projeto

A tabela abaixo resume onde cada capítulo da fase aparece no sistema:

Capítulo	Tema	Aplicação no projeto
2	Lógica booleana e simplificação	Operadores AND/OR/NOT, mintermos, decisão por regras combinadas
3	Subalgoritmos, default, recursividade	Funções modularizadas, validação booleana, propagação de alertas
4	Tabela hash e árvore	Hierarquia da colônia em dict aninhado, acesso O(1) por ID
5	Vetores e matrizes	Listas Python como séries históricas de sensores
6	Arquitetura e ciclo de execução	Loop buscar → decodificar → executar como motor do sistema
7	Regressão linear simples	Previsão eólica por mínimos quadrados, com R^2
8	Fontes renováveis	Potência eólica $P = 0.5 * \rho * A * v^3$, Betz, cut-in/cut-out

Tabela 1 — Capítulos da fase aplicados ao projeto

3. Organização dos dados

Os dados da colônia são organizados aplicando o capítulo 4 (estruturas avançadas). A informação é guardada em três tipos de estrutura:

- **Listas Python** como vetores 1D (cap. 5) — armazenam as séries históricas dos sensores (vento, irradiância, geração);
- **Dicionários (tabela hash)** — acesso médio O(1) por chave; o ID do sensor mapeia diretamente para seus dados;
- **Hierarquia em árvore** — dicionários aninhados representam a estrutura: energético → solar/eólico; ambiental; operacional → módulos; sensores.

A função `acessar(colonia, caminho)` percorre a árvore via uma lista de chaves. A função `buscar_sensor(c, id)` demonstra o acesso $O(1)$ por ID. A função didática `funcao_hash(k, m)` implementa $h(k) = (\text{soma_códigos}(k)) \bmod m$, exatamente como no cap. 4.

4. Lógica booleana e simplificação algébrica (cap. 2)

A lógica de decisão aplica os operadores da álgebra de Boole — AND, OR, NOT — implementados explicitamente em Python. O sistema gera **tabelas-verdade automaticamente** e consegue **extrair expressões por mintermos** (soma de produtos), seguindo a regra do cap. 2.9: para cada linha com $X=1$, variável em 1 entra direta e variável em 0 entra negada (apóstrofe).

O exemplo de simplificação do cap. 3.4.5 do PDF é reproduzido e verificado numericamente:

```
X = (A AND NOT B) OR (A AND B)
X = A AND (NOT B OR B) # fatoracao
X = A AND 1 # complemento: NOT B OR B = 1
X = A # identidade: A AND 1 = A
```

Essa simplificação justifica a estrutura compacta da função `decidir_acao()`, que combina critérios em ordem de prioridade:

- **Crítico** — energia < 30 **AND** consumo alto → *Modo de Emergência*;
- **Alto** — tempestade **AND** energia < 50 → *Modo de Economia*;
- **Médio** — energia < 50 → *Reduzir Consumo*;
- **Normal** — caso contrário → *Manter Sistemas Normais*.

5. Subalgoritmos e recursividade (cap. 3)

Todo o código é organizado em **funções pequenas e nomeadas**, conforme o princípio do cap. 3: cada subalgoritmo resolve apenas um problema. As funções usam **parâmetros default** (ex.: `analisar_energia(geracao, consumo, reserva=0)`) e funções booleanas de validação (`modelo.do_notaValida()` do capítulo).

A **recursividade** aparece em `propagar_alerta()`, que percorre os setores da colônia chamando a si mesma. Seguindo o cap. 3, tem **condição de parada** (índice fora do range) e **argumento que evolui** (índice + 1):

```
def propagar_alerta(setores, indice=0, mensagem='ALERTA'):
    if indice >= len(setores):
        return []
    cabeca = f"[{mensagem}] -> setor '{setores[indice]}'"
    return [cabeca] + propagar_alerta(setores, indice + 1, mensagem)
```

6. Ciclo BUSCAR → DECODIFICAR → EXECUTAR (cap. 6)

O motor do sistema é um ciclo inspirado na máquina de von Neumann (cap. 6). Cada iteração equivale a uma *instrução* da colônia:

- **BUSCAR** — função `buscar(colonia)` lê os sensores e devolve um dicionário com as leituras atuais;
- **DECODIFICAR** — função `decodificar(leituras, colonia)` aplica a análise de energia, a lógica booleana, a previsão por regressão e as fórmulas físicas;
- **EXECUTAR** — função `executar(leituras, info)` escolhe a ação final e prioriza os módulos.

7. Previsão por regressão linear (cap. 7)

A previsão da geração eólica usa **regressão linear simples pelo método dos mínimos quadrados**, implementado manualmente sem bibliotecas. As fórmulas exatas do capítulo 7 são aplicadas:

```
beta1 = sum( (xi - x_med) * (yi - y_med) ) / sum( (xi - x_med)^2 )
beta0 = y_med - beta1 * x_med
y_hat = beta0 + beta1 * x
R2 = 1 - sum((yi - y_hat_i)^2) / sum((yi - y_med)^2)
```

Exemplo aplicado:

```
Entrada: vento = [8, 10, 12, 13, 15]
         geracao = [18, 21, 25, 27, 31]
Reta: energia = 1.622 * vento + 4.961 (R2 = 0.997)
Para vento = 11 -> energia estimada ~ 22.81
```

O R^2 próximo de 1 mostra que o modelo linear representa muito bem a relação entre vento e geração, tornando a previsão confiável para apoiar as decisões da colônia. O sistema também calcula os **resíduos** de cada ponto, permitindo análise de qualidade do ajuste conforme o capítulo.

8. Geração de energia renovável (cap. 8)

Além da regressão estatística, o sistema simula a geração de energia usando as **fórmulas físicas exatas do capítulo 8**:

- **Potência eólica:** $P = 0.5 * \rho * A * v^3$, com $C_p \leq 16/27 \approx 0,593$ (limite de Betz);
- **Energia solar:** $P = \text{irradiância} \cdot \text{área} \cdot \text{eficiência}$;
- **Curva de operação:** cut-in / nominal / cut-out, implementada com if/elif/else;
- **Escolha de painel** (mono, poli, filme fino) conforme prioridade — eficiência, custo ou peso.

Para a atmosfera marciana, usa-se $\rho \sim 0.020 \text{ kg/m}^3$ (cerca de 1/60 da densidade na Terra), o que reduz significativamente a potência captável. A função `classificar_operacao_turbina(v, ...)` informa o estado da turbina em cada faixa de velocidade.

9. Resumo dos resultados por cenário

A lógica de decisão foi testada em quatro cenários representativos:

Cenário	Entrada	Decisão (saída)
---------	---------	-----------------

Energia crítica e consumo alto	energia=25, consumo=70	ATIVAR MODO DE EMERGÊNCIA
Tempestade e energia baixa	energia=45, tempestade=Sim	ATIVAR MODO DE ECONOMIA
Energia baixa, sem tempestade	energia=48, consumo=35	REDUZIR CONSUMO
Condições normais	energia=90, consumo=50	MANTER SISTEMAS NORMAIS

Tabela 2 — Decisões do sistema por cenário

10. Como o sistema melhora o uso de energia

O sistema melhora o uso de energia na colônia em quatro frentes:

- **Identificação antecipada de risco** — a análise compara geração e consumo a cada ciclo e emite ALERTA assim que o consumo ultrapassa a geração, antes que a reserva se esgote;
- **Eliminação de desperdício** — quando a geração supera o consumo, o sistema sugere armazenar o excedente nas baterias;
- **Priorização automática** — em modo de economia ou emergência, módulos não essenciais (estufa, laboratório) são desligados, mas suporte à vida e comunicação ficam sempre ligados;
- **Antecipação por previsão** — a regressão linear estima a geração eólica futura a partir do vento, e a simulação física (cap. 8) informa em qual região da curva de operação a turbina está, permitindo decisões preventivas antes de períodos de baixa produção.

11. Estrutura do projeto e repositório

O sistema é entregue como 6 notebooks Jupyter (.ipynb) autossuficientes em Python puro:

- `00_main.ipynb` — notebook principal que integra tudo;
- `01_dados_colonia.ipynb` — hierarquia e tabela hash (caps. 4, 5);
- `02_logica_booleana.ipynb` — AND/OR/NOT, mintermos e simplificação (cap. 2);
- `03_previsao.ipynb` — regressão linear manual com R^2 (cap. 7);
- `04_energia.ipynb` — geração solar e eólica com Betz e curva de operação (cap. 8);
- `05_decisoes.ipynb` — decisões e recursividade (caps. 2 e 3).

Link do repositório: <https://github.com/<usuario>/aurora-siger-cap1>

Observação: substitua o link acima pela URL real do repositório criado pela equipe.

12. Conclusão

O sistema desenvolvido cumpre o objetivo do desafio: transforma dados brutos dos sensores em decisões autônomas, claras e seguras, evoluindo a colônia de um modelo apenas reativo para um modelo preditivo. Ao integrar todos os conceitos da fase — lógica booleana, subalgoritmos, estruturas avançadas, vetores, ciclo de execução, regressão linear e energia renovável — em uma única solução em Python puro, o projeto demonstra que problemas complexos podem ser modelados com lógica simples, organização clara e interpretação rigorosa dos dados, sem depender de bibliotecas avançadas.